COMS-W4995-011
Neural Rendering for Computer Graphics
Dr. Corey Toler-Franklin
Columbia University

Neural Volume Rendering
DUE: October 30th, 11:59 pm

Overview | Details | Resources | Getting Help | Submitting | Acknowledgements

Overview

Collaboration Rules: You may complete this project individually, or in groups of two or three. You may discuss concepts between groups, but **each group must submit their own original code**.

In project one, you gained experience with novel view synthesis, ray sampling, and point sampling. In project 2, use your sampling code as a starting point to build a volume renderer and optimize it for a basic implicit volume. You will then optimize a neural radiance light field (NeRF). This project is based on a project designed by Shubham Tulsiani at Carnegie Mellon University.

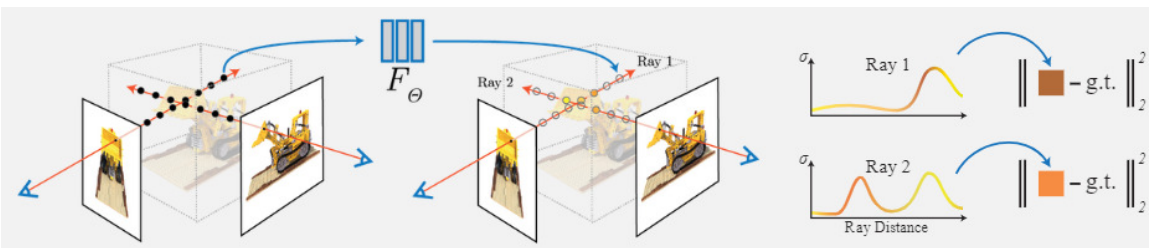

Figure 1: Neural Radiance Fields synthesize views by querying 5D coordinates along camera rays and use classic volume rendering techniques to project the output colors and densities into an image. Source [MST*21]

.

Details

Setup Your Code Base: See project1-docs.pdf and project2-docs.pdf for files referenced in this project. Use AWS (provided), or other cloud computing service, or your personal computer. Everyone in the course has AWS access, and pre-installed APIs like PyTorch, and support libraries (like OpenCV image loading). Remember that AWS access and any data on our storage space **will be removed without notice on December** 12th. This project will be developed using Python, and will require Anaconda or similar virtual environment to install dependencies. Although python and is supported on Windows, macOS and Linux, we recommend that windows users install Virtual Box and work with Python in a Linux environment. See Python installation information here.

Setup and code base instructions from project one

Use the virtual environment and datasets provided in project one.
Ray Sampling and Point Sampling: Use your implementation from project one.

Differentiable Volume Rendering: Project one implements the first stages of a volumetric ren-

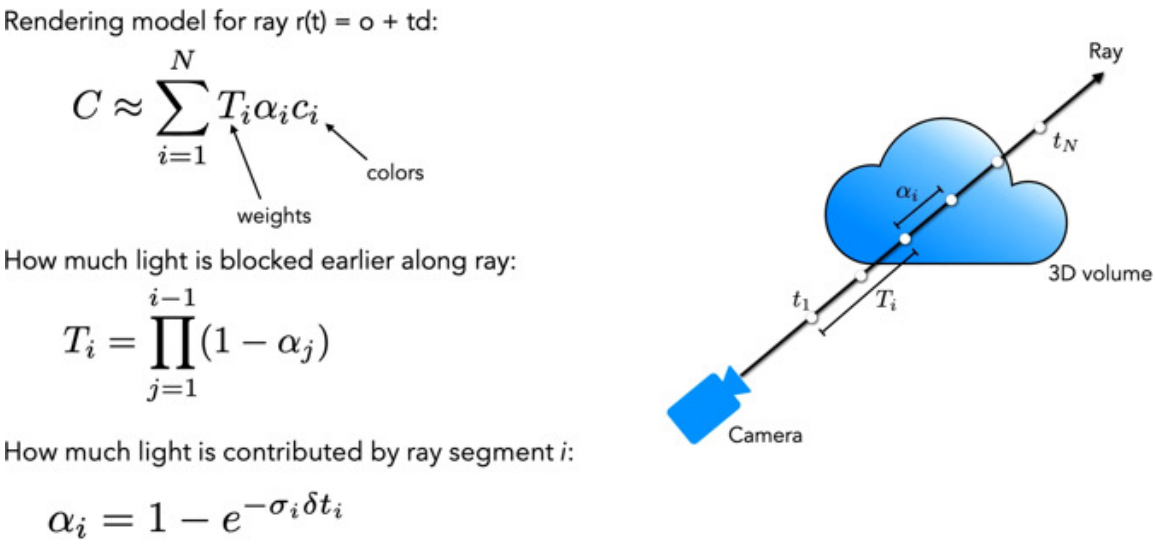

Figure 2: Ray casting. Image source - Ben Mildenhall 2020

derer. You will now complete your volume renderer. Recall in class that we compute the output color in a volume renderer by calculating the accumulated color along a ray cast through the volume (Figure 2). Each sample point along the ray contributes its emission color while being attenuated by its absorption coefficient using a process called "ray marching" and applying the Beer-Lambert law to model the absorption effect. In this emission-absorption (EA) model, volumes are typically described by their appearance (e.g. emission) and geometry (absorption) at every point in 3D space. In project two you will implement a differentiable renderer for EA volumes.

Recall the code structure.

There are four major components of our differentiable volume rendering pipeline:

1. **The camera**: pytorch3d.CameraBase
2. **The scene**: SDFVolume in implicit.py
3. **The sampling routine**: StratifiedSampler in sampler.py
4. **The renderer**: VolumeRenderer in renderer.py

StratifiedSampler provides a method for sampling multiple points along a ray traveling through the scene (also known as raymarching). Together, a sampler and a renderer describe a rendering pipeline. Like traditional graphics pipelines, this rendering procedure is independent of the scene and camera.

The scene sampler and renderer are all together under the Model class in volume_rendering_main.py. In particular the Model's forward method invokes a VolumeRenderer instance with a sampling strategy and volume as input.

The RayBundle class in ray_utils.py provides a convenient wrapper around several inputs to the volume rendering procedure per ray.

In order to perform rendering, you will implement the following routines:

1. Ray sampling from cameras: you will fill out methods in ray_utils.py to generate world space rays from a particular camera. (proj1)
2. Point sampling along rays: you will fill out the StratifiedSampler class to generate sample points along each world space ray (proj1)
3. Rendering: you will fill out the VolumeRenderer class to evaluate a volume function at each sample point along a ray, and aggregate these evaluations to perform rendering. (proj2)

Volume Rendering

Implement volume rendering. With the configs/box.yaml configuration, we provide you with an SDFVolume instance describing a box. You can check out the code for this function in implicit.py, which converts a signed distance function into a volume. If you want, you can even implement your own SDFVolume classes by creating new signed distance function class, and adding it to sdf_dict in implicit.py. Take a look at this great web page here for formulas for some simple/complex SDFs.

Implementation

1. VolumeRenderer._compute_weights and
2. VolumeRenderer._aggregate.
3. You will also modify the VolumeRenderer.forward method to render a depth map in addition to color from a volume

From each volume evaluation you will get both volume density, and a color:

```
# Call implicit function with sample points
implicit_output = implicit_fn(cur_ray_bundle)
density = implicit_output['density']
feature = implicit_output['feature']
```

You will then use the following equation to render color along a ray:

$$L(x,\omega) = \sum_{i-1}^{N} T(x,x_{t_i})(1-e^{-\sigma_{t_i}\Delta t})L_e(x_{t_i},\omega)$$

where σ is density, Δt is the length of current ray segment, and L_e is color:

$$T(x,x_{t_i}) = T(x,x_{t_{i-1}})e^{-\sigma_{t_{i-1}}\Delta t}$$

Compute the weights $T*(1-exp(-\sigma*\Delta t))$ in VolumeRenderer._compute_weights, and perform the summation in VolumeRenderer._aggregate. Note that for the first segment $T=1$.

Use weights, and aggregation function to render color and depth (stored in RayBundle.sample_lengths).

Visualization

By default, your results will be written out to images/part_1.gif. **Provide a visualization of the depth in your results folder and in your write-up**. Note that the depth should be normalized by its maximum value. Figure 3 shows the expected result.

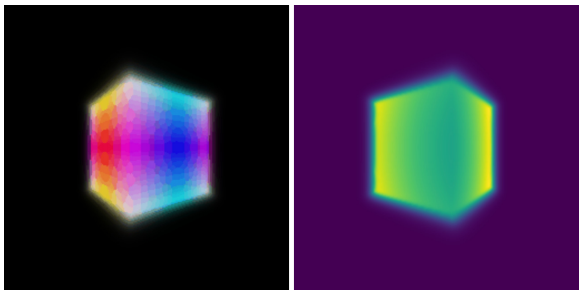

Figure 3: Depth visualization shown right.

Optimizing a basic implicit volume

Random ray sampling

Since you have now implemented a differentiable volume renderer, we can use it to optimize the parameters of a volume! We have provided a basic training loop in the train method in volume_rendering_main.py.

Depending on how many sample points we take for each ray, volume rendering can consume a lot of memory on the GPU (especially during the backward pass of gradient descent). Because of this, it usually makes sense to sample a subset of rays from a full image for each training iteration. In order to do this, implement the get_random_pixels_from_image method in ray_utils.py, invoked here:

xy_grid = get_random_pixels_from_image(cfg.training.batch_size, image_size, camera) # TODO: implement in ray_utils.py

Loss and training

Replace the loss in train

loss = None

with mean squared error between the predicted colors and ground truth colors rgb_gt.

python volume_rendering_main.py –config-name=train_box

This will optimize the position and side lengths of a box, given a few ground truth images with known camera poses (in the data folder). Report the center of the box, and the side lengths of the box after training, rounded to the nearest $\frac{1}{100}$ decimal place.

Visualization

The code renders a spiral sequence of the optimized volume in images/part_2.gif. Compare this gif to the image below (compare your results to the gif file can be found in the project2-support folder), **save the gif file in your results folder and attach it in your write-up**:

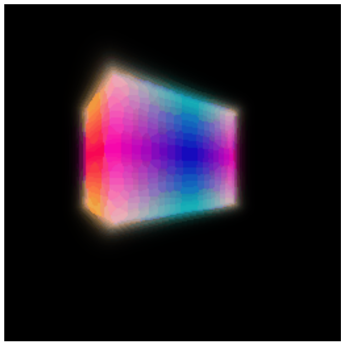

Figure 4: Volume optimization.

Optimizing a Neural Radiance Field (NeRF)

Now implement an implicit volume as a Multi-Layer Perceptron (MLP) in the NeuraRadianceField class in implicit.py. This MLP should map 3D position to volume density and color. Specifically:

1. Your MLP should take in a RayBundle object in its forward method, and produce color and density for each sample point in the RayBundle.
2. You should also fill out the loss in train_nerf in the volume_rendering_main.py file.

You will then use this implicit volume to optimize a scene from a set of RGB images. We have implemented data loading, training, checkpointing for you, but this part will still require you to do a bit more legwork than for Parts 1 and 2. You will have to write the code for the MLP yourself — feel free to reference the NeRF paper, though you should not directly copy code from an external repository.

Implementation

Here are a few things to note:

1. For now, your NeRF MLP does not need to handle view dependence, and can solely depend on 3D position.
2. You should use the ReLU activation to map the first network output to density (to ensure that density is non-negative).
3. You should use the Sigmoid activation to map the remaining raw network outputs to color
4. You can use Positional Encoding of the input to the network to achieve higher quality. We provide an implementation of positional encoding in the HarmonicEmbedding class in implicit.py.

Visualization

Train a NeRF on the lego bulldozer dataset with:

python volume_rendering_main.py –config-name=nerf_lego

This will create a NeRF with the NeuralRadianceField class in implicit.py and use it as the implicit_fn in VolumeRenderer. It will also train a NeRF for 250 epochs on 128x128 images.

You can also modify the experimental settings in configs/nerf_lego.yaml — though the current settings should allow you to train a NeRF on low-resolution inputs in a reasonable amount of time. After training, a spiral rendering will be written to images/part_3.gif. **Save the NeRF lego results gif file in your results folder and add attach it to your report**. It should look something like this (compare your results to the gif file found in the project2-support folder):

Figure 5: Neural Randiance Fields: Lego.

Extra Credit 10 pts

Add view dependence to your NeRF model! Specifically, make it so that emission can vary with viewing direction. You can read NeRF or other papers for how to do this effectively — if you're not careful, your network may overfit to the training images. Discuss the trade-offs between increased view dependence and generalization quality.

While you may use the lego scene to test your code, employ the materials scene to show the results of your method (experimental settings can be found in nerf_materials.yaml and nerf_materials_highres.yaml). **Save your results in the results folder and include in your write-up**

Explain your code and run it for us: Make a 5-7 minute video screen tour walking us through your code, explaining what you did in your own words. Run your code and show us the program generating results at the command line or web interface. We suggest programs for doing this in CourseWorks. You may however use any program you like. These are only suggestions.

In your video you must show us how to navigate your code:

- general structure
- functions for volume rendering with implicit optimization
- Loss function
- NeRF Optimization

Grading: You will be evaluated based on the following criteria (1) source code correctness and completeness 20%, (2) volume rendering 25% (3) basic implicit optimization: randm ray sampling 10% and (4) loss and training 10% (5) optimizing a NeRF 25% (6) written report (all other items not scored in 2-5) 10%.

Resources

Instruction documents referenced in this assignment will be posted to CourseWorks.

All students in the class will have access to the AWS.

Remember to back-up your work regularly!!! Use version control to store your work. DO NOT PUBLISIZE SOLUTIONS.

Getting Help

Email to me (ct3219@columbia.edu) or the TA (kr3131@columbia.edu@columbia.edu) with issues or questions.

Office Hours are posted on CourseWorks.

Collaborating You may discuss this assignment with each other but each group should submit individual work. Remember to always credit outside sources you use in your code. The Columbia University Honor Code must be followed.

Submitting

Only one submission required per-group. Non submitting group members should add the list of group members as a note in the CourseWorks assignment. Upload one zip file to CourseWorks. If your zip file is too large to upload, you must submit your written report to CourseWorks by the due date and upload your zip file to a file share (e.g. ONEDRIVE) and email the link to the instructor by the due date. All submissions (CourseWorks or otherwise) must be completely uploaded by the due date and time. This assignment is subject to the late policy for course assignments.

Your submission should include everything needed to test, run and understand your code including:

- Complete source code with any third party libraries not available in the AMI distributions.
- gif file of depth visualization from volume rendering before optimization.
- gif file of spiral sequence of optimized implicit volume.
- gif file of Nerf lego.
- (optional) gif file showing view dependent effects with materials dataset.
- Any other input needed to run your code that is not mentioned in this list.
- Written report that includes:
  - An explanation of your implementation as requested in bold in this document.
  - Screen shots of results requested (do not forget to also submit the original files).
  - Instructions on how to run your programs.
  - Anything you would like us to know (bugs, difficulties).
- 5-7 minute video screen tour as decribed in this document.

References

[MST*21] MILDENHALL B., SRINIVASAN P. P., TANCIK M., BARRON J. T., RAMAMOORTHI R., NG R.: Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM 65*, 1 (2021), 99–106.

Acknowledgements

Sources and Datasets:
ShapeNet, shapenet.org
Neural Radiance Fields [MST*21]
Shubham Tulsiani, Carnegie Mellon: Learning for 3D Vision

©Corey Toler-Franklin 2024, Columbia University, Barnard College